

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа 2

Выполнил:

Цатинян Артём

Группа К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Тема лабораторного проекта:

Приложение для бронирования столиков в ресторанах

1. Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению
2. Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend
3. Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
4. Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

Ход работы

1. Функциональные требования

1.1 Аутентификация и управление профилем

- Пользователь может зарегистрироваться, указав email, password, first_name, last_name и phone
- Пользователь может авторизоваться с использованием email и password
- После успешного входа система выдает JWT-токен
- Пользователь может просматривать информацию своего профиля
- Пользователь может редактировать данные своего профиля

1.2 Работа с ресторанами

- Пользователь может получать список ресторанов
- Должна поддерживаться фильтрация ресторанов по кухне, городу, ценовому сегменту и количеству гостей
- Пользователь может просматривать подробную информацию о ресторане
- Пользователь может получать список доступных временных слотов для бронирования
- Пользователь может просматривать меню ресторана
- Пользователь может просматривать фотографии ресторана
- Пользователь может просматривать отзывы о ресторане

1.3 Работа с бронированиями

- Авторизованный пользователь может создавать бронирование
- При создании бронирования система проверяет корректность временного интервала, вместимость столика и отсутствие пересечений с другими бронированиями
- Пользователь может просматривать историю своих бронирований
- Пользователь может просматривать детали конкретного бронирования
- Пользователь может отменять собственное бронирование

1.4 Работа с отзывами

- Авторизованный пользователь может оставлять отзыв по факту завершенного бронирования
- Пользователь может просматривать список отзывов ресторана

- Для одного бронирования может быть оставлен только один отзыв
- Администратор может редактировать и удалять отзывы в рамках модерации

1.5 Административные функции

- Администратор может создавать, просматривать, изменять и удалять пользователей
- Администратор может создавать, просматривать, изменять и удалять кухни
- Администратор может создавать, просматривать, изменять и удалять рестораны
- Администратор может создавать, просматривать, изменять и удалять столики ресторана
- Администратор может создавать, просматривать, изменять и удалять категории меню
- Администратор может создавать, просматривать, изменять и удалять позиции меню
- Администратор может создавать, просматривать, изменять и удалять фотографии ресторанов
- Администратор может просматривать все бронирования и изменять их параметры и статус

2. Нефункциональные требования

- API реализуется в формате REST
- Обмен данными между клиентом и сервером выполняется в формате

JSON

- Спецификация API оформляется в формате OpenAPI 3.0 и может быть открыта в Swagger UI / Swagger Editor
- Для защищенных методов используется авторизация по Bearer JWT Token
- Пароли пользователей хранятся только в виде хэша
- Все основные сущности системы имеют уникальные идентификаторы типа UUID
- Для списков ресторанов, бронирований, пользователей и отзывов должна поддерживаться пагинация
- Система должна обеспечивать целостность данных при создании бронирований
- Время в API передается в формате ISO 8601
- API должен возвращать предсказуемые HTTP-статусы и единый формат ошибок
- Для административных методов должна использоваться ролевая модель доступа
- Архитектура API должна быть пригодна для последующей реализации на Java и Spring Boot

3. Формат API

Выбран формат REST API.

REST API подходит для данного проекта, так как предметная область

естественно описывается через ресурсы: users, restaurants, bookings, reviews, cuisines, menu_categories, menu_items, restaurant_tables,

restaurant_photos. Для каждого ресурса определены URL, HTTP-методы,

тела запросов и ответов, а также коды ошибок.

Спецификация API была подготовлена в формате OpenAPI, что позволяет

использовать Swagger для просмотра и обсуждения спроектированных эндпоинтов.

4. Спроектированные эндпоинты

Основные группы эндпоинтов:

4.1 Аутентификация

- POST /auth/register
- POST /auth/login

4.2 Профиль пользователя

- GET /users/me
- PATCH /users/me

4.3 Пользователи (административные методы)

- GET /users
- POST /users
- GET /users/{userId}
- PATCH /users/{userId}
- DELETE /users/{userId}

4.4 Кухни

- GET /cuisines
- POST /cuisines
- GET /cuisines/{cuisineId}

- PATCH /cuisines/{cuisineId}
- DELETE /cuisines/{cuisineId}

4.5 Рестораны

- GET /restaurants
- POST /restaurants
- GET /restaurants/{restaurantId}
- PATCH /restaurants/{restaurantId}
- DELETE /restaurants/{restaurantId}
- GET /restaurants/{restaurantId}/availability
- GET /restaurants/{restaurantId}/menu
- GET /restaurants/{restaurantId}/reviews

4.6 Столики ресторана

- GET /restaurants/{restaurantId}/tables
- POST /restaurants/{restaurantId}/tables
- GET /restaurants/{restaurantId}/tables/{tableId}
- PATCH /restaurants/{restaurantId}/tables/{tableId}
- DELETE /restaurants/{restaurantId}/tables/{tableId}

4.7 Категории меню

- GET /restaurants/{restaurantId}/menu-categories
- POST /restaurants/{restaurantId}/menu-categories
- GET /restaurants/{restaurantId}/menu-categories/{menuCategoryId}
- PATCH /restaurants/{restaurantId}/menu-categories/{menuCategoryId}
- DELETE /restaurants/{restaurantId}/menu-categories/{menuCategoryId}

4.8 Позиции меню

- GET /restaurants/{restaurantId}/menu-items
- POST /restaurants/{restaurantId}/menu-items
- GET /restaurants/{restaurantId}/menu-items/{menuItemId}
- PATCH /restaurants/{restaurantId}/menu-items/{menuItemId}
- DELETE /restaurants/{restaurantId}/menu-items/{menuItemId}

4.9 Фотографии ресторанов

- GET /restaurants/{restaurantId}/photos
- POST /restaurants/{restaurantId}/photos
- GET /restaurants/{restaurantId}/photos/{photoId}
- PATCH /restaurants/{restaurantId}/photos/{photoId}
- DELETE /restaurants/{restaurantId}/photos/{photoId}

4.10 Бронирования

- GET /bookings
- POST /bookings
- GET /bookings/me
- GET /bookings/{bookingId}
- PATCH /bookings/{bookingId}
- PATCH /bookings/{bookingId}/cancel

4.11 Отзывы

- POST /restaurants/{restaurantId}/reviews
- GET /reviews/{reviewId}

- PATCH /reviews/{reviewId}
- DELETE /reviews/{reviewId}

5. Тела запросов и ответов

Для каждого эндпоинта в OpenAPI-схеме определены:

- параметры пути и query-параметры
- JSON-тело запроса
- JSON-тело ответа
- обязательные и необязательные поля
- форматы данных
- коды успешных и ошибочных ответов

Примеры сущностей, описанных в схеме:

- UserProfile
- Cuisine
- RestaurantSummary
- RestaurantDetails
- RestaurantTable
- MenuCategory
- MenuItem
- Booking
- Review
- ErrorResponse

Также в схеме описаны специальные request-модели:

- RegisterRequest

- LoginRequest
- UpdateProfileRequest
- RestaurantUpsertRequest
- RestaurantTableUpsertRequest
- MenuCategoryUpsertRequest
- MenuItemUpsertRequest
- RestaurantPhotoUpsertRequest
- CreateBookingRequest
- AdminUpdateBookingRequest
- CreateReviewRequest
- ReviewUpdateRequest

6. Обработка ошибок

Основные коды ошибок:

- 400 Bad Request — некорректные параметры или тело запроса
- 401 Unauthorized — отсутствует или неверный токен
- 403 Forbidden — недостаточно прав доступа
- 404 Not Found — ресурс не найден
- 409 Conflict — конфликт бизнес-логики, например пересечение бронирований или дублирование значения
- 422 Unprocessable Entity — логическая ошибка запроса
- 500 Internal Server Error — внутренняя ошибка сервера

Для единообразной обработки ошибок используется общая структура `ErrorResponse`, содержащая статус, текст ошибки, сообщение, временную метку, путь запроса и дополнительные детали.

Вывод

В результате выполнения домашней работы были описаны функциональные и нефункциональные требования приложения, выбран формат REST API, спроектированы основные и административные эндпоинты, а также продуманы структуры запросов, ответов и возможные ошибки.